Equipo 3

Decisiones técnicas y backlog

Decisiones técnicas:

Propuesta: Blazor Server + Aspire + PostgreSQL + Keycloak, desplegado en Azure Container Apps.

Decisión	Opciones	Propuesta	Por qué
Tecnología web	Blazor / MVC	Blazor Server	Trae SignalR integrado: el monitoreo en tiempo real sale casi sin código extra. Componentes reutilizables para tablas y dashboards.
Arquitectura	Monolítica / modular / microservicios con Aspire	Aspire con pocos servicios: web, API, simulador	El simulador del inversor es naturalmente un servicio aparte (en la realidad es otro hardware). Aspire levanta todo con un comando, trae panel de logs y métricas, y suma por investigar algo nuevo. Se evita fragmentar más para no complicar.
Nube	Sí / no	Sí, Azure Container Apps	La alta disponibilidad exige varias réplicas y despliegue sin corte (revisiones con traspaso de tráfico). Aspire despliega ahí con `azd up`. Azure for Students da crédito gratis.
Proveedor OpenID	Keycloak / Auth0 / Entra ID	Keycloak	Gratuito, open source, corre en contenedor dentro de Aspire, maneja roles (Administrador, Técnico). Auth0 queda como plan B si Keycloak complica el deploy.
Base de datos	SQL Server / PostgreSQL / otras	PostgreSQL + EF Core	Gratis, buen soporte en Aspire y Azure. Para las lecturas del simulador se puede sumar TimescaleDB (series de tiempo) sin cambiar de motor.

Lenguaje natural	Azure OpenAI / API de Claude / OpenAI	A definir	Solo consultas: el modelo traduce la pregunta a llamadas de solo lectura sobre la API.

Glosario

Tecnología web

- **MVC (Model-View-Controller):** cada acción del usuario hace un pedido al servidor y este devuelve una página HTML nueva. Es el modelo clásico.
- **Blazor:** framework de Microsoft para hacer interfaces web con C# en lugar de JavaScript. La interfaz se arma con *componentes*, piezas reutilizables como una tabla, una tarjeta o un gráfico.
- **Blazor Server:** la lógica de la interfaz corre en el servidor. El navegador mantiene una conexión abierta y recibe los cambios al instante, sin recargar la página.
- **SignalR:** librería de .NET para comunicación en tiempo real entre servidor y navegador. El servidor *empuja* datos sin que el usuario los pida. Blazor Server ya la usa por dentro, y por eso el monitoreo en vivo sale casi gratis.

Arquitectura

- **Monolítica:** toda la aplicación es un solo proyecto que se despliega junto.
- **Modular:** sigue siendo una sola aplicación, pero dividida internamente en módulos bien separados (Parques, Mantenimiento, Monitoreo).
- **Microservicios:** la aplicación se parte en varios programas independientes que se comunican por red. Cada uno se despliega por separado.
- **Aspire (.NET Aspire):** herramienta de Microsoft para orquestar aplicaciones con varios servicios. Con un comando levanta todo junto: la web, la API, la base de datos y Keycloak. Además trae un *dashboard* para ver logs (mensajes que escribe cada servicio) y métricas (uso de CPU, tiempos de respuesta, errores).
- **API:** el servicio que expone la lógica y los datos para que otros programas, como la web, los consuman.
- **Simulador (mock):** programa que imita al inversor real y devuelve datos inventados pero realistas.

Nube y disponibilidad

- **Nube:** correr la aplicación en servidores de un proveedor (Azure, AWS) en vez de en una máquina propia.
- **Contenedor (Docker):** un paquete con la aplicación y todo lo que necesita para correr. Funciona igual en cualquier máquina.
- **Azure Container Apps:** servicio de Azure que ejecuta contenedores y se encarga de escalarlos y actualizarlos.

- **Réplicas:** varias copias de la misma aplicación corriendo a la vez. Si una se cae o se está actualizando, las otras siguen atendiendo.
- **Despliegue sin corte:** se sube una versión nueva sin que el sistema deje de estar disponible. En Container Apps, cada versión es una *revisión*. El tráfico pasa de la vieja a la nueva recién cuando la nueva está lista.
- **`azd up`:** comando de Azure Developer CLI que crea los recursos en Azure y despliega la aplicación en un paso.
- **Azure for Students:** cuenta gratuita para estudiantes con crédito para usar Azure, sin tarjeta de crédito.

Autenticación

- **OpenID Connect (OIDC):** estándar para que una aplicación delegue el login en un sistema externo. La app no maneja contraseñas: el usuario se loguea en el proveedor, y este le devuelve a la app un *token* (un comprobante firmado) que dice quién es y qué roles tiene.
- **Auth0:** proveedor OpenID comercial en la nube, con plan gratuito.
- **Entra ID:** el proveedor de identidad de Microsoft (antes Azure AD).
- **Open source:** software de código abierto y gratuito.
- **Roles:** permisos agrupados. Por ejemplo, el *Administrador* puede hacer los ABM y el *Técnico* solo ve sus mantenimientos.

Base de datos

- **SQL Server / PostgreSQL:** motores de base de datos relacionales. PostgreSQL es gratuito y open source.
- **Series de tiempo:** datos que son mediciones con fecha y hora, como las lecturas de los paneles cada X segundos. Crecen muchísimo y se consultan por rangos de tiempo.
- **TimescaleDB:** extensión de PostgreSQL optimizada para series de tiempo. Hace rápidas consultas como "generación por mes" sobre millones de lecturas.

Lenguaje natural

- **LLM (modelo de lenguaje):** IA que entiende y genera texto, como GPT o Claude.
- **Azure OpenAI / OpenAI / API de Claude:** distintas formas de usar un LLM desde código, pagando por uso. Lo usarían para traducir "¿cómo está funcionando todo hoy?" a consultas sobre la API.

Keycloak (Unica cosa que me genera duda, discutir si dejarla o sacarla)

Es un servidor de identidad open source: el sistema que maneja usuarios, contraseñas, login y roles. En vez de programar el login ustedes:

1. El usuario entra a SolarUY y la app lo redirige a la pantalla de login de Keycloak.
2. Se loguea ahí, y Keycloak lo devuelve a SolarUY con un token.
3. SolarUY lee el token y sabe quién es y qué rol tiene, sin haber tocado nunca su contraseña.

Cumple con el requisito de la letra, que pide autenticación *externa* con estándar OpenID. Y como corre en un contenedor, Aspire lo levanta junto con el resto. En Keycloak se crea un *realm* (un espacio para la app), se definen los roles y se dan de alta los usuarios.

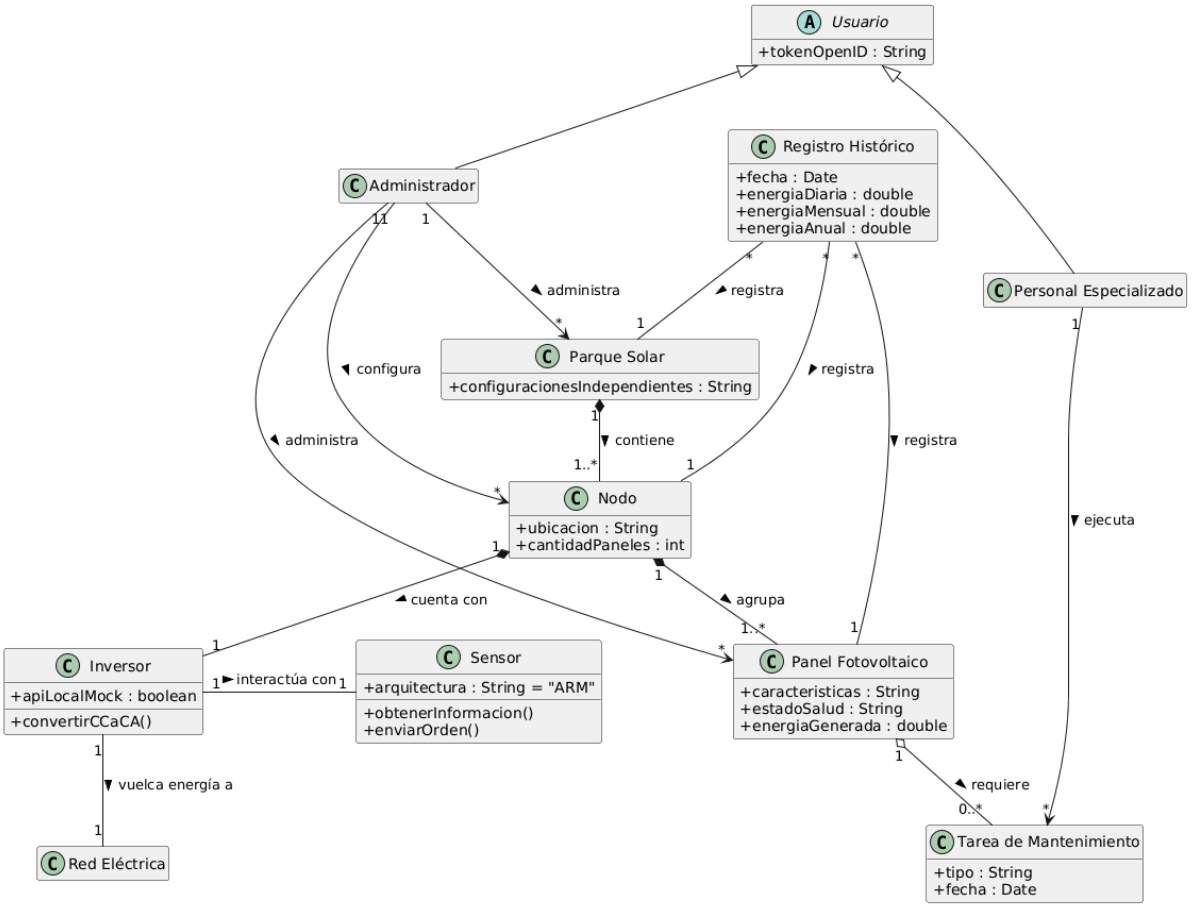
Configurar EF Core

EF Core (Entity Framework Core) es el ORM de .NET. Permite trabajar con la base de datos usando clases de C# en lugar de escribir SQL a mano. Por ejemplo, ustedes escriben `context.Parques.Add(parque)` y EF Core genera el `INSERT`.

Configurarlo implica:

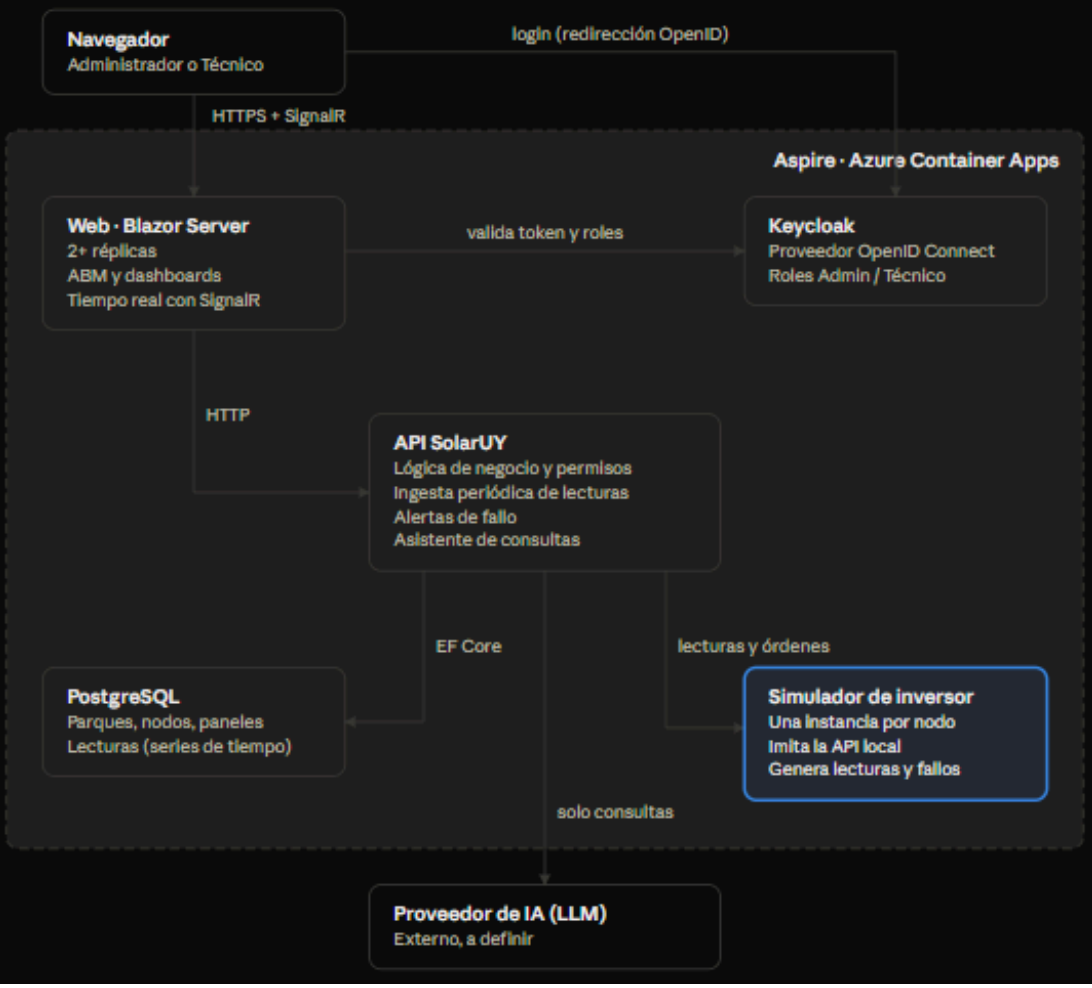
1. **Instalar los paquetes** de EF Core y el proveedor de PostgreSQL (Npgsql).
2. **Crear las entidades:** clases `Parque`, `Nodo`, `Panel`, etc., que se convierten en tablas.
3. **Crear el `DbContext`:** la clase que representa la base y lista las tablas.
4. **Definir relaciones:** por ejemplo, que un nodo pertenece a un parque o que un panel tiene muchas lecturas.
5. **Conectar a la base:** la cadena de conexión, que con Aspire se inyecta sola.
6. **Crear migraciones:** archivos generados que crean o modifican las tablas cuando cambia el modelo (`dotnet ef migrations add Inicial`).

Modelo de Dominio - SolarUY



Arquitectura

Todo corre dentro de Aspire; el simulador ocupa el lugar del inversor real



Funcionalidades

40 funcionalidades en 9 módulos.

Módulo	Funcionalidad	Quién
Usuarios	Iniciar Sesión	Todos
Usuarios	Cerrar Sesión	Todos
Usuarios	Gestionar Roles de Usuario	Administrador
Usuarios	Alta Técnico	Administrador
Usuarios	Listar Técnicos	Administrador
Parques	Alta Parque	Administrador
Parques	Baja Parque	Administrador
Parques	Modificar Parque	Administrador
Parques	Listar Parques	Todos
Nodos	Alta Nodo	Administrador
Nodos	Baja Nodo	Administrador
Nodos	Modificar Nodo	Administrador

Nodos	Listar Nodos	Todos
Nodos	Asignar Inversor a Nodo	Administrador
Paneles	Alta Panel	Administrador
Paneles	Baja Panel	Administrador
Paneles	Modificar Panel	Administrador
Paneles	Listar Paneles	Todos
Paneles	Asignar Panel a Nodo	Administrador
Mantenimiento	Reservar Fecha Mantenimiento	Administrador
Mantenimiento	Asignar Mantenimiento a Técnico	Administrador
Mantenimiento	Cancelar Mantenimiento	Administrador
Mantenimiento	Listar Mantenimientos	Administrador
Mantenimiento	Ver Mis Mantenimientos	Técnico
Mantenimiento	Confirmar Mantenimiento	Técnico
Monitoreo	Ver Generación en Tiempo Real del Parque	Todos

Monitoreo	Ver Generación en Tiempo Real del Nodo	Todos
Monitoreo	Ver Generación en Tiempo Real del Panel	Todos
Monitoreo	Ver Estado de Salud de Paneles	Todos
Monitoreo	Recibir Alerta de Falla	Todos
Control del inversor	Activar Panel	Administrador
Control del inversor	Desactivar Panel	Administrador
Control del inversor	Activar Inyección a la Red	Administrador
Control del inversor	Desactivar Inyección a la Red	Administrador
Históricos	Consultar Generación por Día	Todos
Históricos	Consultar Generación por Mes	Todos
Históricos	Consultar Generación por Año	Todos
Históricos	Consultar en Lenguaje Natural	Todos
Simulador	Simular Lecturas del Inversor	Sistema
Simulador	Simular Fallas de Paneles	Sistema

Repositorio en Github:
https://github.com/AgustinMS2/SolarUY